*FINAL ASSESSMENT PROJECT*

# Module Details

| | |
|---|---|
| **Module Code** | UFCFK1-15-0 |
| **Module Title** | Fundamentals of Data Science |
| **Module Tutors** | Saurav Gautam |
| **Year** | 2025/2026 |
| **Component/Element Number** | Portfolio |
| **Weighting** | 40% |

# Dates

| | |
|---|---|
| **Submission Date** | **17-April-2026** |
| **Submission Place** | Backboard |
| **Submission Time** | 23:59 |
| **Submission Notes** | Submit Gitlab URL |


**Team Members:**

- **Adarsha Karki**
- **Bishesh Thapa**
- **Jonisha Ghimire**


**Github URL:** [https://github.com/Ads4331X/student-management-system](https://github.com/Ads4331X/student-management-system)

# Introduction

Student Profile Management System is a command line application developed as the final assignment of Fundamental of Data Science. This system manages student personal information, examination grades (5 subjects) and ECA activities using four CSV files to store data.

## Objectives

- Build a login system with username and password validation and role-based access
- Allow admins to perform full CRUD operations on all student data
- Allow students to view and update their own information
- Demonstrate OOP concepts: classes, inheritance, and encapsulation
- Store all data in CSV files using pandas for file handling
- Implement an analytics dashboard using pandas, numpy, and matplotlib

## Technologies Used

**Python** (Main programming language)

**Pandas** (CSV file handling, DataFrames, grade calculations)

**Numpy** (Statistical analysis such as mean, std, correlation, trend line)

**Matplotlib** (Bar charts, scatter plots, pie charts, grade charts)

# System Design

## OOP Class Structure

The system is built using Object Oriented Programming. The User class acts as the base class, and both Student and Admin inherit from it. This avoids repeating common fields like user_id, username, full_name, email, phone, and address.

| Class | Inherits From | Role in System |
|---|---|---|
| User | — | Base class. Holds all common fields and to_dict() / display() methods. |
| Student | User | Role fixed to 'student'. Used for all student accounts. |
| Admin | User | Role fixed to 'admin'. Used for admin accounts. |
| AuthSystem | — | Handles login(), register_student(), and delete_user(). |
| AdminPanel | — | All admin operations: CRUD on students, grades, ECA, insights. |
| StudentPanel | — | Student operations: view profile, grades, ECA, update profile, chart. |
| AnalyticsDashboard | — | matplotlib dashboard and performance alerts. |
| FileHandler | — | All CSV read/write using pandas. |

## File Structure

The project follows a modular structure where each file has one clear responsibility:

```
student_system/
  main.py            — Entry point, login loop, menu routing
  auth.py            — Login validation, registration, deletion
  models.py          — OOP classes: User, Student, Admin
  admin.py           — Admin panel: all CRUD operations
  student.py         — Student portal: view and update own data
  analytics.py       — analytics dashboard and alerts
  file_handler.py    — All CSV file I/O using pandas
  utils.py           — Banner, colours, clear screen helpers
  data/
    users.csv        — All user accounts
    passwords.csv    — Usernames and passwords
    grades.csv       — Marks for 5 subjects per student
    eca.csv          — Extracurricular activities
```

## Data File Formats

All data is stored as CSV files. Pandas reads and writes these files throughout the system.

**users.csv:**

It consists of seven columns and they are
user_id, username, full_name, email, role, phone, address.

```
users.csv > data
...
user_id,username,full_name,email,role,phone,address
A001,admin,System Administrator,admin@email.edu,admin,9800000000,test Office
S001,hari,Hari don,hari@email.edu,student,9800000001,Kathmandu
S002,shya,Shyam dev,shya.@email.edu,student,9800000002,Pokhara
S003,ram,Ram Bahadur,ram@email.edu,student,9800000003,Lalitpur
You, 35 minutes ago • edited test data …
```

**passwords.csv:**

It consists of two columns and they are
username, password.

```
passwords.csv > data
Ads4331x, 2 days ago | 2 authors (Ads4331x and one other)
username,password
admin,admin123
ram,ram123
ram,ram123
```

**grades.csv:**

It consists of six columns and they are
student_id, mathematics, science, english, programming, data_science

```
> grades.csv > data
Ads4331x, 2 days ago | 1 author (Ads4331x)
student_id,mathematics,science,english,programming,data_science
S001,78.0,85.0,72.0,90.0,88.0
S002,92.0,88.0,95.0,87.0,91.0
S003,55.0,60.0,58.0,52.0,65.0
```

**eca.csv:**

It consist of two columns and they are

student_id, activities (semicolon-separated)

```
> eca.csv > data
Ads4331x, 2 days ago | 1 author (Ads4331x)
student_id,activities
S001,Football Club;Debate Society;Coding Club
S002,Chess Club;Science Society
S003,Music Band
```

# Features and Implementation

## Login System

The login system is implemented in auth.py via the AuthSystem.login() method. The system allows a maximum of 3 attempts before returning the user to the main menu.

**How it works step by step:**

- User enters username and password
- System reads passwords.csv: checks if username exists
- Compares entered password against stored password
- If match, reads users.csv and returns the User object
- Checks user.role: routes to admin_menu() or student_menu()
- If wrong: shows error, increments attempt counter
- After 3 failed attempts: returns to main menu

## Admin Panel

The AdminPanel class in admin.py provides nine operations. All file changes go through file_handler.py.

List of operations:

1. **Add student:**
   Uses the function add_student() which collects details, validates uniqueness, saves new student in all four CSV files
2. **View students:**
   Uses the function view_all_students() which loads and displays all students in a formatted table from users.csv
3. **Update profile:**
   Uses the function update_student() which finds student by ID, updates their information from users.csv
4. **Delete student:**
   Uses the function delete_student() which removes data from all files requires YES confirmation
5. **Update grades:**
   Uses the function update_grades() which shows current marks, validates it and saves new marks in grades.csv
6. **Update ECA:**
   Uses the function update_eca() which add, remove, or clear ECA activities for a student in eca.csv
7. **Grade insights:**
   Uses the function view_insights() which reads grades.csv and eca.csv, calculates subject averages using NumPy, ranks students using Pandas, and shows ECA activity counts for each student
8. **Dashboard:**
   Uses the function show_dashboard() which reads all CSV files and displays a 4-panel chart using Matplotlib, including bar charts, scatter plot, and pie chart for overall analysis

9. **Alerts:**
   Uses the function performance_alerts() which reads grades.csv and eca.csv, identifies students below 60% average, and displays their weak subjects with suggested improvements

```
STUDENT PROFILE MANAGEMENT SYSTEM

ADMIN PANEL — System Administrator

[1] Add New Student
[2] View All Students
[3] Update Student Profile
[4] Delete Student
[5] Update Student Grades
[6] Update Student ECA
[7] View Grade Insights
[8] Analytics Dashboard
[9] Performance Alerts
[0] Logout

Choice:
```

## Student Portal

The StudentPanel class in student.py provides five operations. Students can view their own data and update limited profile details.

List of operations:

1. **View profile:**
   Uses the function view_profile() which reads users.csv and displays the student's personal information using display()
2. **View grades:**
   Uses the function view_grades() which reads grades.csv, calculates average and grade letter, and displays marks with a text-based bar chart
3. **View ECA:**
   Uses the function view_eca() which reads eca.csv and displays the student's extracurricular activities
4. **Update profile:**
   Uses the function update_profile() which allows the student to update email, phone, and address in users.csv
5. **Grade chart:**
   Uses the function view_grade_chart() which reads grades.csv and displays a color-coded bar chart using Matplotlib

```
STUDENT PROFILE MANAGEMENT SYSTEM

STUDENT PORTAL — Ram Bahadur

[1] View My Profile
[2] View My Grades
[3] View My ECA
[4] Update My Profile
[5] View My Grade Chart
[0] Logout

Choice:
```

# Concepts Demonstrated

- **Inheritance:**
  Student and Admin classes inherit from the User base class, allowing reuse of common attributes such as user_id, username, and email.
- **Encapsulation:**
  File operations are encapsulated within file_handler.py, meaning all CSV read/write logic is handled in one place and hidden from other modules.
- **Classes and Objects:**
  The system is built using multiple classes including User, Student, Admin, AuthSystem, AdminPanel, StudentPanel, and AnalyticsDashboard, each responsible for a specific part of the system.
- **Methods:**
  Each class contains well-defined methods such as to_dict(), display(), login(), and register_student() to perform specific tasks.
- **Exception Handling:**
  try-except blocks are used in file operations and user input handling to prevent crashes and ensure the program runs smoothly even when errors occur.

# Analytics Dashboard (Task 2)

The AnalyticsDashboard class in analytics.py implements the Task 2 bonus. It is accessible from the admin menu as options 8 (Dashboard) and 9 (Performance Alerts). It uses pandas, numpy, and matplotlib throughout.

## How Each Library is Used

**Pandas:**
Used to load grades.csv into a DataFrame and perform data manipulation. It calculates student averages, maps ECA activity counts, and filters students who are below the required performance level.

**NumPy:**
Used for statistical analysis such as calculating mean, standard deviation, maximum, and minimum values for each subject. It also computes correlation between ECA participation and grades, generates a trend line, and identifies failing subjects.

**Matplotlib:**
Used for data visualization. It creates a 4-panel dashboard including bar charts for subject and student averages, a scatter plot showing correlation with a trend line, and a pie chart for grade distribution. It is also used to generate individual student grade charts.

## Dashboard Panels

The dashboard displays four visual panels to represent student performance using different analytical views.

- **Panel 1: Subject Averages Bar Chart**
  Displays average marks for each subject. Bars are colour-coded based on performance: green (≥70), orange (≥50), and red (<50). A pass mark reference line is also shown for comparison.
- **Panel 2: Student Average Rankings**
  Shows a horizontal bar chart of all students ranked by their average marks, with a reference line indicating the pass mark.
- **Panel 3: ECA vs Grades Scatter Plot**
  Visualises the relationship between ECA participation and academic performance. Each student is represented as a point, and a trend line is fitted using numpy.polyfit().

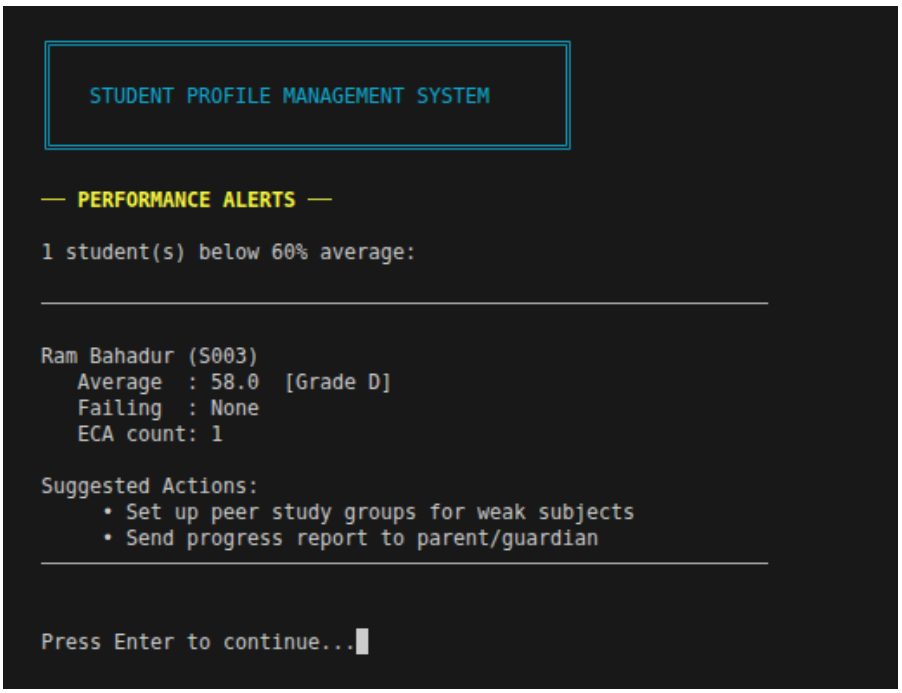
- **Panel 4: Grade Distribution Pie Chart**
  Displays the overall grade distribution (A, B, C, D, F) across all students to show performance spread.

### Performance Alerts

The performance_alerts() method identifies students whose average is below 60% (Grade D and below). For each at-risk student the system:

- Calculates their average using pandas mean()
- Uses numpy.where() to find which subjects have marks below 50%
- Prints name, average, grade letter, and list of failing subjects
- Suggests personalised actions: academic counselling if below 50%, peer study groups if 50-60%, ECA encouragement if no activities

With the sample data, Ram Bahadur (S003) has an average of 58.0 (Grade D) and is correctly flagged as at-risk.




## Testing and Sample Output

The following test cases were carried out to verify all system features. Screenshots of actual program output are included below each test.

### Login System

| Test | Input | Expected Output | Result |
|---|---|---|---|
| Valid admin login | admin / admin123 | Welcome, System Administrator! [ADMIN] | Pass |
| Valid student login | john / john123 | Welcome, John Doe! [STUDENT] | Pass |
| Wrong password | john / wrong123 | Incorrect username or password. | Pass |

| Test | Input | Expected Output | Result |
|---|---|---|---|
| Unknown username | nobody / pass | Incorrect username or password. | Pass |
| 3 failed attempts | wrong x3 | Too many failed attempts. | Pass |

**Output:**

**Valid admin login:**

```
STUDENT PROFILE MANAGEMENT SYSTEM

— LOGIN —

Attempt 1/3
Username: admin
Password: admin123

Welcome, System Administrator! [ADMIN]
Press Enter to continue...
```

**Wrong credentials (error message):**

```
STUDENT PROFILE MANAGEMENT SYSTEM

— LOGIN —

Attempt 1/3
Username: ad
Password: ad
Incorrect username or password.

Attempt 2/3
Username:
```

**Three failed attempts:**

```
STUDENT PROFILE MANAGEMENT SYSTEM

— LOGIN —

Attempt 1/3
Username: ad
Password: ad
Incorrect username or password.

Attempt 2/3
Username: 123
Password: ram123
Incorrect username or password.

Attempt 3/3
Username: adas
Password: asdasd
Incorrect username or password.

Too many failed attempts.
Press Enter to return to main menu...
```

**Valid student login:**

```
STUDENT PROFILE MANAGEMENT SYSTEM

— LOGIN —

Attempt 1/3
Username: ram
Password: ram123

Welcome, Ram Bahadur! [STUDENT]
Press Enter to continue...
```

## Admin (Add Student)

| Test | Input | Expected Output | Result |
|---|---|---|---|
| Add valid student | S0102, ads, ads test, ... | Student 'ads test' added successfully. | Pass |
| Duplicate ID | S0102 (existing) | ID 'S0102' already exists. | Pass |
| Duplicate username | john (existing) | Username 'john' already exists. | Pass |
| Empty required field | Leave name blank | All fields except phone/address required. | Pass |

**Output:**

**Add a valid student:**

```
STUDENT PROFILE MANAGEMENT SYSTEM

— ADD NEW STUDENT —
Student ID (e.g. S004): S0102
Username               : ads
Full Name              : ads test
Email                  : ads.test1@gmail.com
Phone                  : 000000000
Address                : test,test2
Password               : ads
Student 'ads test' added successfully.

Press Enter to continue...
```

**Duplicate ID:**

```
STUDENT PROFILE MANAGEMENT SYSTEM

— ADD NEW STUDENT —
Student ID (e.g. S004): S0102
Username               : duplicateID
Full Name              : test dublicateID
Email                  : test@gmail.com
Phone                  : 1234567890
Address                : dup,test
Password               : 1234
ID 'S0102' already exists.

Press Enter to continue...
```

**Duplicate Username:**

```
STUDENT PROFILE MANAGEMENT SYSTEM

— ADD NEW STUDENT —
Student ID (e.g. S004): S1234
Username               : ads
Full Name              : ada test
Email                  : ado@gmail.com
Phone                  : 0192019491
Address                : 12,12
Password               : 12f1
Username 'ads' already exists.

Press Enter to continue...
```

**All Empty field:**

```
STUDENT PROFILE MANAGEMENT SYSTEM

— ADD NEW STUDENT —
Student ID (e.g. S004):
Username              :
Full Name             :
Email                 :
Phone                 :
Address               :
Password              :
All fields except phone/address are required.

Press Enter to continue...
```

## Admin (Update and Delete)

| Test | Input | Expected Output | Result |
|---|---|---|---|
| Update student name | S001, new name | Student updated successfully. | Pass |
| Delete (confirm YES) | S004, YES | Student deleted from all 4 CSVs. | Pass |
| Delete (cancel) | S004, NO | Deletion cancelled. | Pass |
| Delete unknown ID | S999 | Student 'S999' not found. | Pass |

**Output:**

**Delete (Yes):**

```
— DELETE STUDENT —
Enter Student ID to delete: S0102

About to delete: ads test (S0102)
Type YES to confirm: YES
Student 'ads test' deleted.

Press Enter to continue...
```

**Delete (cancel):**

```
— DELETE STUDENT —
Enter Student ID to delete: S001

About to delete: John Doe (S001)
Type YES to confirm: no
Deletion cancelled.

Press Enter to continue...
```

**Delete unknown student ID:**

```
— DELETE STUDENT —
Enter Student ID to delete: 1234
Student '1234' not found.

Press Enter to continue...
```

**Before Update (user.csv):**

```
data > users.csv > data
You, 2 hours ago | 2 authors (Ads4331x and one other)
user_id,username,full_name,email,role,phone,address
A001,admin,System Administrator,admin@email.edu,admin,9800000000,test Office
S001,john,John Doe,john@email.edu,student,9800000001,Kathmandu
S002,jane,Jane Smith,jane@email.edu,student,9800000002,Pokhara
S003,ram,Ram Bahadur,ram@email.edu,student,9800000003,Lalitpur
S0102,ads,ads test,ads.test1@gmail.com,student,000000000,"test,test2"
You, 2 hours ago • edited test data …
```

**Update Student Name:**

```
STUDENT PROFILE MANAGEMENT SYSTEM

— UPDATE STUDENT PROFILE —
Enter Student ID: S001

ID       : S001
Name     : John Doe
Username : john
Email    : john@email.edu
Phone    : 9800000001
Address  : Kathmandu
Role     : Student

(Press Enter to keep current value)

Full Name  [John Doe]: Jackel Dev
Email      [john@email.edu]: jackel@yahoo.com
Phone      [9800000001]: 0010020034
Address    [Kathmandu]: Nepalgunj

Student updated successfully.

Press Enter to continue...
```

**After Update (user.csv):**

```
data > users.csv > data
You, 2 hours ago | 2 authors (You and one other)
user_id,username,full_name,email,role,phone,address
A001,admin,System Administrator,admin@email.edu,admin,9800000000,test Office
S001,john,Jackel Dev,jackel@yahoo.com,student,0010020034,Nepalgunj
S002,jane,Jane Smith,jane@email.edu,student,9800000002,Pokhara
S003,ram,Ram Bahadur,ram@email.edu,student,9800000003,Lalitpur
S0102,ads,ads test,ads.test1@gmail.com,student,000000000,"test,test2"
```

## Admin — Grades and ECA

| Test | Input | Expected Output | Result |
|---|---|---|---|
| Update grades valid | S001, 85 for maths | Grades updated successfully. | Pass |
| Grade out of range | 150 | Must be between 0 and 100. | Pass |
| Grade non-numeric | abc | Enter a valid number. | Pass |
| Add ECA activity | S001, Chess Club | Added: Chess Club | Pass |
| Remove ECA activity | Remove item 1 | Removed: Football Club | Pass |

**Output:**

**Update grades (valid):**

```
STUDENT PROFILE MANAGEMENT SYSTEM

— UPDATE GRADES —
Enter Student ID: S001

Current grades for John Doe:

Subject              Current
──────────────────────────────
Mathematics             78.0
Science                 85.0
English                 72.0
Programming             90.0
Data_science            88.0

Enter new marks (0-100). Press Enter to keep current.

Mathematics [78.0]: 85
Science [85.0]:
English [72.0]:
Programming [90.0]:
Data_science [88.0]:

Grades updated successfully.

Press Enter to continue...
```

**Grade out of range:**

```
STUDENT PROFILE MANAGEMENT SYSTEM

— UPDATE GRADES —
Enter Student ID: S001

Current grades for John Doe:

Subject              Current
──────────────────────────────
Mathematics            85.0
Science                85.0
English                72.0
Programming            90.0
Data_science           88.0

Enter new marks (0-100). Press Enter to keep current.

Mathematics [85.0]: 150
Must be between 0 and 100.
Mathematics [85.0]:
```

**Non numeric input:**

```
STUDENT PROFILE MANAGEMENT SYSTEM

— UPDATE GRADES —
Enter Student ID: S001

Current grades for John Doe:

Subject              Current
──────────────────────────────
Mathematics            85.0
Science                85.0
English                72.0
Programming            90.0
Data_science           88.0

Enter new marks (0-100). Press Enter to keep current.

Mathematics [85.0]: Aer
Enter a valid number.
Mathematics [85.0]:
```

**Add ECA activities:**

```
STUDENT PROFILE MANAGEMENT SYSTEM

— UPDATE ECA —
Enter Student ID: S001

ECA for John Doe:
[1] Football Club
[2] Debate Society
[3] Coding Club

[1] Add activity  [2] Remove activity  [3] Clear all  [0] Cancel

Choice: 1
Activity name: Chess Club
Added: Chess Club

Press Enter to continue...
```

**Remove  ECA activity:**

```
STUDENT PROFILE MANAGEMENT SYSTEM

— UPDATE ECA —
Enter Student ID: S001

ECA for John Doe:
[1] Football Club
[2] Debate Society
[3] Coding Club
[4] Chess Club

[1] Add activity  [2] Remove activity  [3] Clear all  [0] Cancel

Choice: 2
Enter number to remove: 1
Removed: Football Club

Press Enter to continue...
```

## Admin (Grade Insights)

The grade insights screen uses numpy for subject statistics and pandas for student ranking.

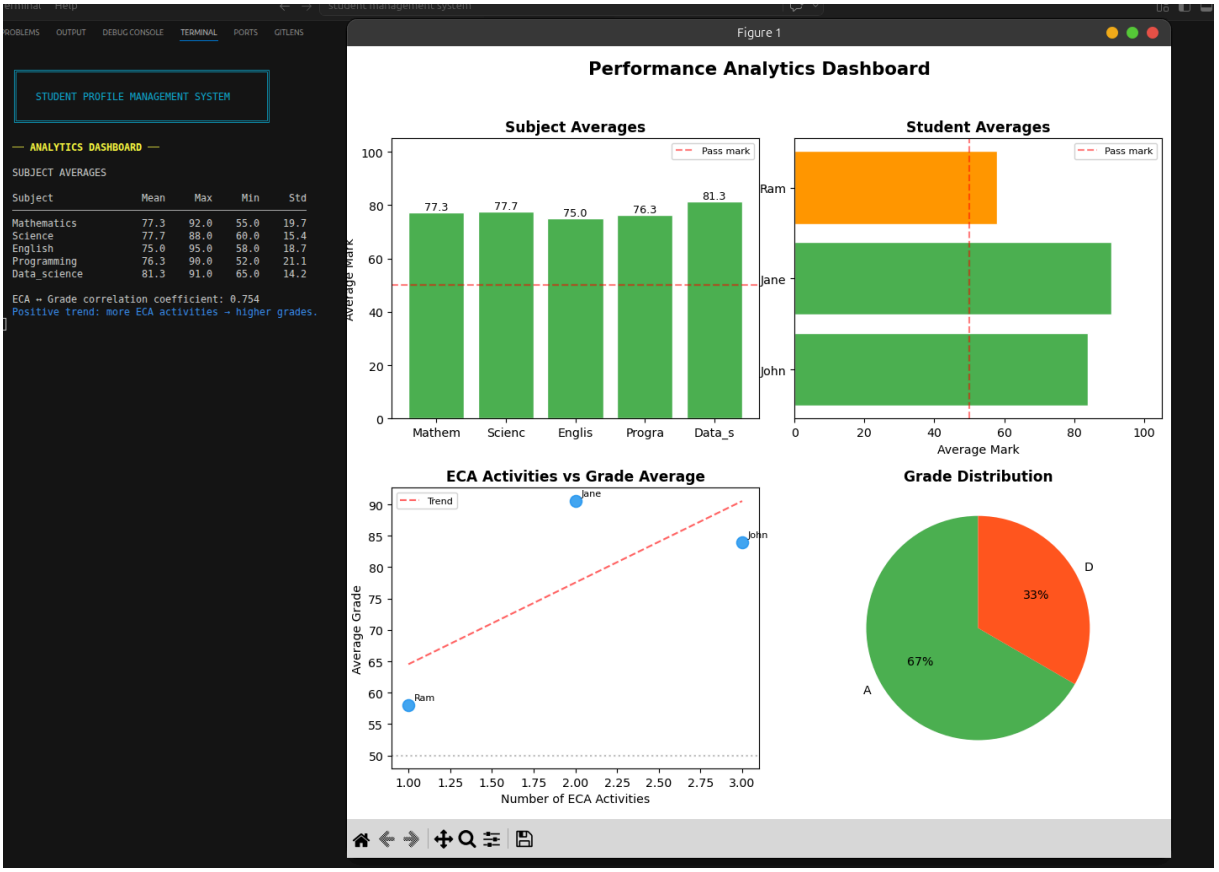
**Output:**

```
STUDENT PROFILE MANAGEMENT SYSTEM

— GRADE INSIGHTS —

SUBJECT STATISTICS (numpy)

Subject              Average   Highest   Lowest   Std Dev
Mathematics             77.3      92.0     55.0      16.0
Science                 77.7      88.0     60.0      12.6
English                 75.0      95.0     58.0      15.3
Programming             76.3      90.0     52.0      17.2
Data_science            81.3      91.0     65.0      11.6

STUDENT RANKING (pandas)

Rank   ID       Name                    Average  Grade
1      S002     Jane Smith                 90.6  A
2      S001     John Doe                   84.0  A
3      S003     Ram Bahadur                58.0  D

ECA PARTICIPATION

John Doe              : 3 activities  ●●●
Jane Smith            : 2 activities  ●●
Ram Bahadur           : 1 activities  ●

Press Enter to continue...
```

## Analytics Dashboard

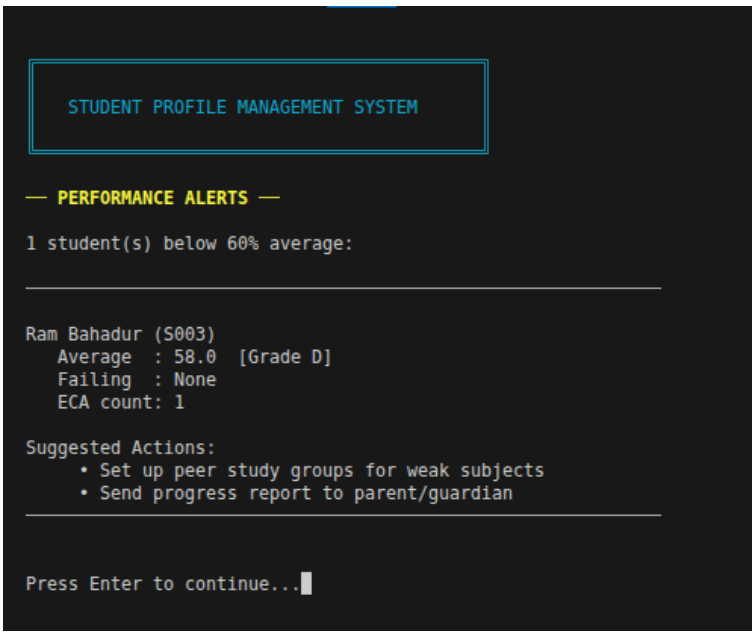
The 4-panel matplotlib dashboard opens as a separate window showing all charts together.

**Output:**




## Performance Alerts

| Test | Expected Output | Result |
|---|---|---|
| Ram avg 58.0 < 60.0 | Ram flagged as at-risk, Grade D, suggested interventions shown | Pass |
| Jane avg 90.6 | Jane is NOT flagged (above threshold) | Pass |
| John avg 82.6 | John is NOT flagged (above threshold) | Pass |

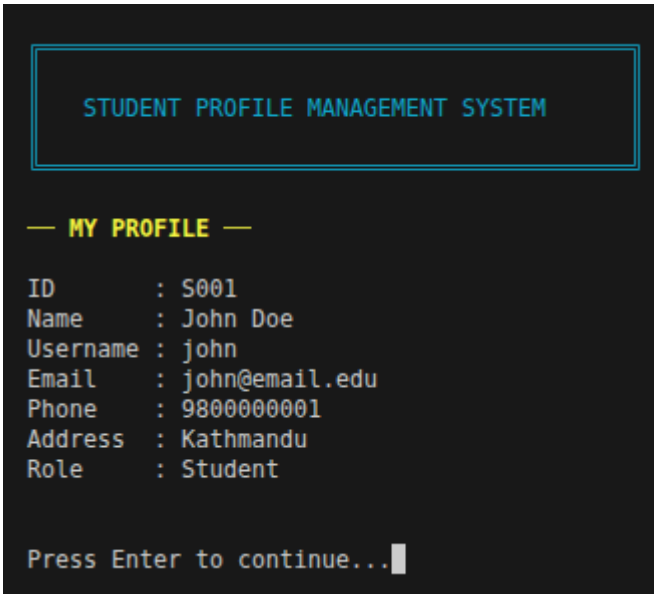


## Student Portal

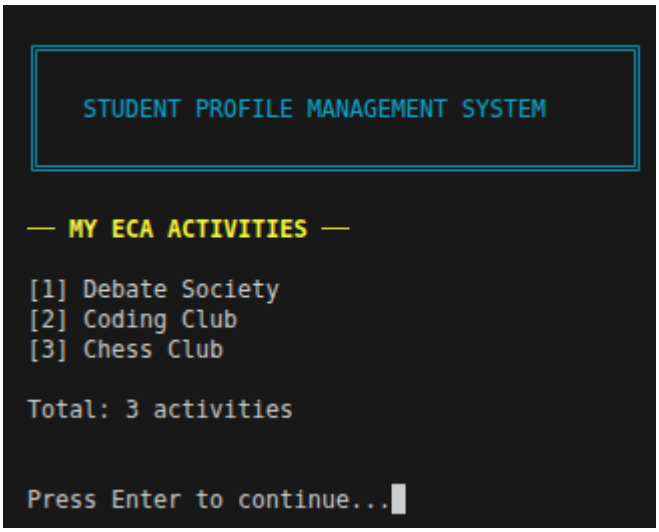
| Test | Input | Expected Output | Result |
|---|---|---|---|
| View profile | Login as john | Shows name, email, phone, address, role | Pass |
| View ECA | Login as john | Lists Football Club, Debate Society, Coding Club | Pass |
| View grades | Login as john | Shows 5 subjects with text bar chart and letter grade | Pass |
| Update profile | New email, address and phone | Profile updated successfully. | Pass |
| View grade chart | Login as john | Opens matplotlib bar chart window with coloured bars | Pass |

**Output:**

**View Profile:**




**View ECA:**



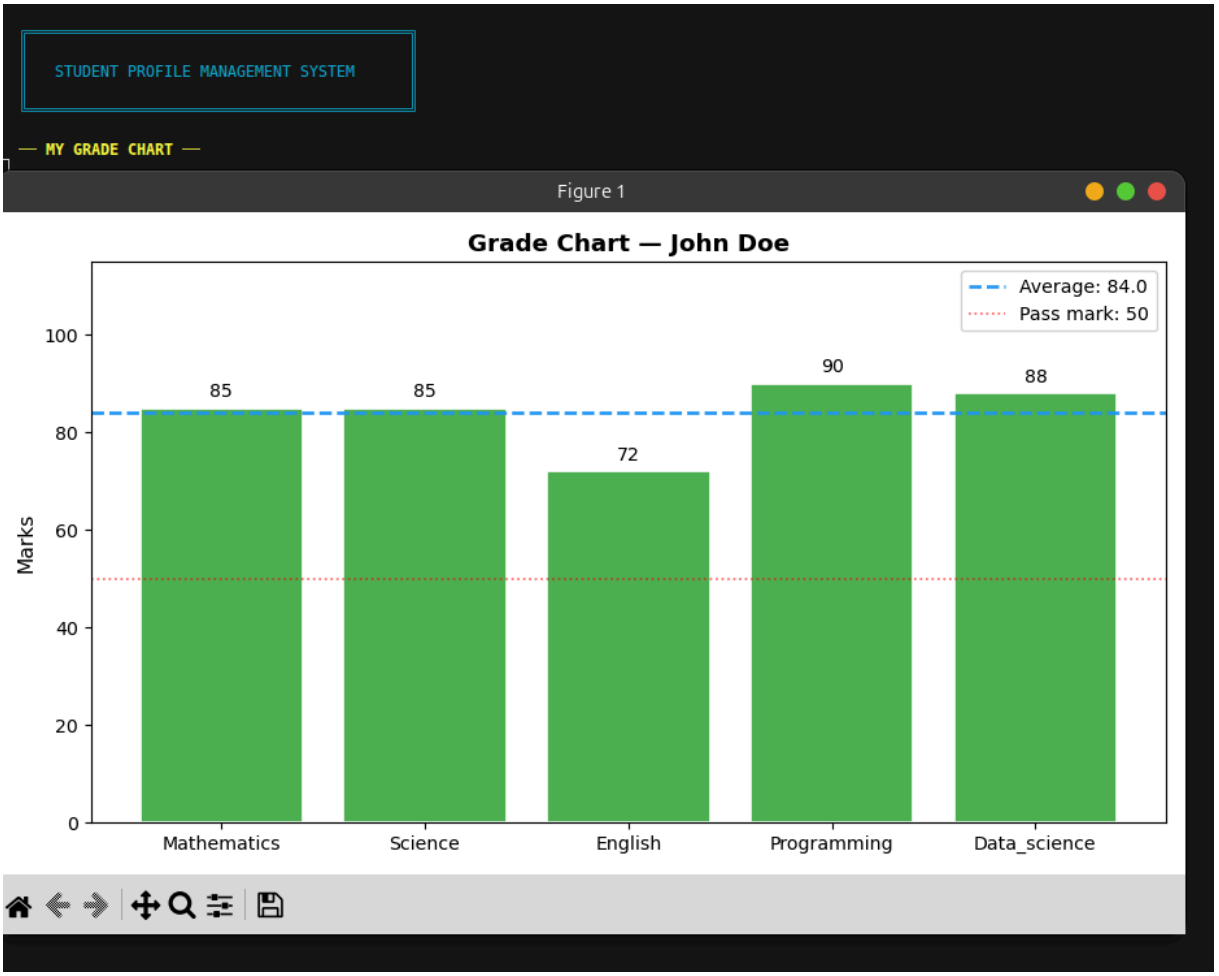
**View Grades:**

```
STUDENT PROFILE MANAGEMENT SYSTEM

— MY GRADES —

Grades for John Doe

Subject              Mark   Bar
------------------------------------------------
Mathematics          85.0
Science              85.0
English              72.0
Programming          90.0
Data_science         88.0
------------------------------------------------
Average              84.0   Grade: A

Press Enter to continue...
```

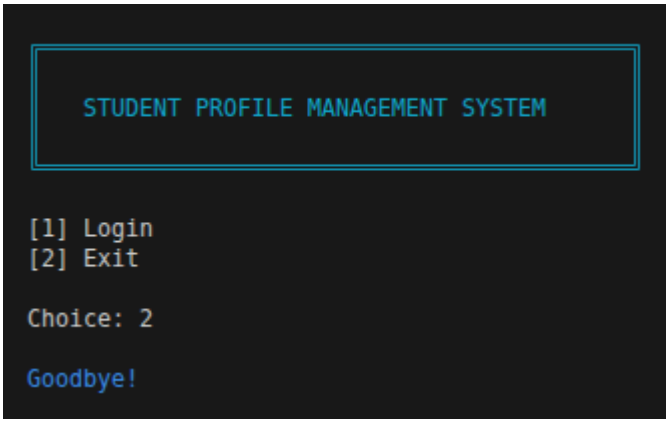
**Update Profile:**

```
STUDENT PROFILE MANAGEMENT SYSTEM

— UPDATE MY PROFILE —

(Press Enter to keep current value)

Email   [john@email.edu]: update@gmail.com
Phone   [9800000001]: 123451345
Address [Kathmandu]: Birgunj

Profile updated successfully.

Press Enter to continue...
```

**View Grade Chart:**


STUDENT PROFILE MANAGEMENT SYSTEM
— MY GRADE CHART —
Figure 1
Grade Chart — John Doe
Average: 84.0
Pass mark: 50
Marks
100
80
60
40
20
0
85
85
72
90
88
Mathematics
Science
English
Programming
Data_science

## Exit:


STUDENT PROFILE MANAGEMENT SYSTEM
[1] Login
[2] Exit
Choice: 2
Goodbye!

## Limitations and Future Work

### Current Limitation

- Passwords are stored in plain text (the assignment brief permits this for learning purposes)
- The system is single-user only (no concurrent access support)
- Matplotlib charts open in a separate window and pause the terminal until closed
- No email format or phone number validation on input fields
- Students cannot change their own password

### Possible Improvements

- Replace CSV files with a SQLite database for better data integrity and concurrent access
- Add password hashing using Python's hashlib library
- Add input validation for email format and phone number
- Build a web interface using Flask so the system runs in a browser
- Allow students to change their own password from the student portal

## Team Contributions and Reflection

### Work Contribution

| Member | Files | What they did |
|---|---|---|
| Adarsha Karki | main.py, utils.py, student.py, analytics.py | Entry point, menu routing, student portal, analytics dashboard and charts. |
| Bishesh Thapa | models.py, file_handler.py | OOP class design (User, Student, Admin) and all CSV file handling using pandas. |
| Jonisha Ghimire | auth.py, admin.py | Login and authentication system and full admin panel with CRUD operations. |

**Adarsha Karki** developed the entry point of the system in main.py, handling the main loop, login routing, and separate menu systems for admin and student roles with a maximum of three login attempts. He built utils.py with all shared display helpers including the banner, color coded messages, and screen clearing. In student.py, he implemented the full student portal including viewing grades with a text based bar chart, viewing ECA activities, updating personal details, and generating a colour-coded matplotlib bar chart with an average line and pass mark reference. He also built the complete analytics dashboard in analytics.py using a 4 panel matplotlib layout with subject averages, student averages, an ECA vs grade scatter plot with a numpy polyfit trend line, and a grade distribution pie chart, along with the performance alerts system using pandas filtering and numpy to identify failing subjects.

**Bishesh Thapa** designed the data structure of the system in models.py, defining a base User class with shared attributes and a to_dict() method for CSV serialisation, then extending it into Student and Admin subclasses using inheritance. In file_handler.py, he implemented all CSV read and write functions for the four data files using pandas, including upsert_student_grades() which inserts a new grade row or updates an existing one without duplicates, and load_eca() which parses semicolon-separated activity strings into Python lists.

**Jonisha Ghimire** built the authentication system in auth.py, validating usernames and passwords against the CSV files, enforcing a three attempt login limit, and handling registration and deletion of users across all four data files. In admin.py, she implemented the full admin panel with nine operations: adding new students with unique ID and username checks, updating student profiles, deleting records from all CSV files simultaneously, updating grades with range validation, and managing ECA activities. She also built the grade insights feature using numpy for subject statistics and pandas for student ranking, and enforced role-based access so only authenticated admins can access these functions.

# Reflection

## Adarsha Karki

This project gave me practical experience in building a complete multi module Python application. Working on main.py taught me how to structure program flow clearly with handling the login loop, enforcing the three attempt limit and routing each user to the correct menu based on their role without any module knowing about the others. Building utils.py as a shared layer also helped me understand why separating display logic from business logic makes the whole codebase easier to maintain.

The hardest part of my work was building the analytics dashboard in analytics.py. Producing a clean four panel layout using plt.subplots(2, 2) required careful spacing adjustments with plt.subplots_adjust, and getting the ECA scatter plot trend line to work correctly using numpy.polyfit and numpy.poly1d took several attempts to get the x range right. The performance alerts feature also required combining pandas filtering with numpy.where to identify exactly which subjects each student was failing, rather than just flagging the average.

Building the student grade chart in student.py taught me how to use colour thresholds meaningfully (green for above 70, orange for above 50, and red below) rather than just producing a plain chart. Working through GitHub with the team showed me how important it is to agree on function return formats early, since my modules depended on what file_handler.py returned. If I had more time I would make the dashboard interactive with user controlled filters.

## Bishesh Thapa

This project helped me understand how object-oriented design and file handling work together in a real application. Designing models.py required thinking carefully about which attributes and methods truly belong in the base User class so that Student and Admin could inherit without duplication. The to_dict() method was more important than it looked. It had to return column names in exactly the right order so that file_handler.py could write rows back to CSV without any mismatches.

The most challenging function I wrote was upsert_student_grades() in file_handler.py. A straightforward append would silently create duplicate rows every time grades were updated, so I had to first check whether the student ID already existed in the DataFrame and either update the row in place using df.loc or concatenate a new row using pd.concat. Getting this right without corrupting the CSV took several careful tests. Similarly, load_eca() required filtering out empty strings after splitting the semicolon separated activity field, otherwise blank activity entries would appear for students with no ECA recorded.

This project showed me how critical the data layer is, every other module in the system depended on what file_handler.py returned, so any mistake in parsing or saving would break features that Adarsha and Jonisha were building at the same time. Coordinating through GitHub made me realise how important it is to agree on data formats before everyone starts coding. In future I would replace CSV storage with SQLite for more reliable querying and updates.

## Jonisha Ghimire

This project was a valuable practical experience in semester 2. Building auth.py taught me how authentication systems need to handle multiple failure modes cleanly such as wrong password, username not found, and file read errors without crashing. Implementing the three attempt login limit required tracking failed attempts carefully across the loop and returning the user to the main menu gracefully after the third failure. Coordinating

with admin.py so that the role check happened before any sensitive operation was exposed taught me how important it is to enforce access control at the right point in the flow.

The most difficult function I implemented was the delete operation in admin.py. Removing a student required deleting their record from users.csv, passwords.csv, grades.csv, and eca.csv simultaneously, and if any one file was left with a dangling record it would cause errors in other parts of the system. Getting all four deletions to happen correctly and in the right order took careful testing. Building the grade insights feature was also challenging as it required using numpy to compute subject-level statistics and pandas to sort and rank students by their average score.

Collaborating through GitHub taught me the value of clear communication and version control when multiple people are editing related modules. I particularly enjoyed seeing the admin panel work end to end, adding a student, updating their grades, and watching the insights and ranking update correctly. This project improved my skills in exception handling, file I/O, and system integration, and has given me much more confidence in building larger Python applications.